my_minsec_stopwatch 설계 보고서

Oct 22, 2026 · @태

1. 개요

Basys3 보드(Artix-7 xc7a35tcpg236-1)에서 분/초 시계와 10 ms 단위 stopwatch를 하나의 FSM으로 전환하며 동작시키고, 일시정지 중에는 4자리 FND에 circle animation을 표시하는 설계를 Verilog로 구현했다. 개발 도구는 Vivado 2020.2이며 시스템 클럭은 100 MHz이다.

버튼 3개(Left, Center, Right)로 모드를 바꾼다. 버튼 입력은 10 ms debounce를 거친 뒤 누르는 순간을 1 클럭 펄스로 만들어 FSM에 넣는다. FND는 1 ms마다 한 자리씩 켜는 다중화(multiplex) 스캔으로 표시한다.

모드	FND 표시	LED[15:14]	버튼 동작
분/초 시계 (CLOCK)	mm.ss	00	Left: stopwatch로 전환
stopwatch (STOPWATCH)	ss.cc (cc는 10 ms 단위)	01	Left: 시계로 복귀, Right: 일시정지, Center: 0으로 초기화
일시정지 (PAUSE)	멈춘 ss.cc 값	10	Right: 재개, Center: 0으로 초기화

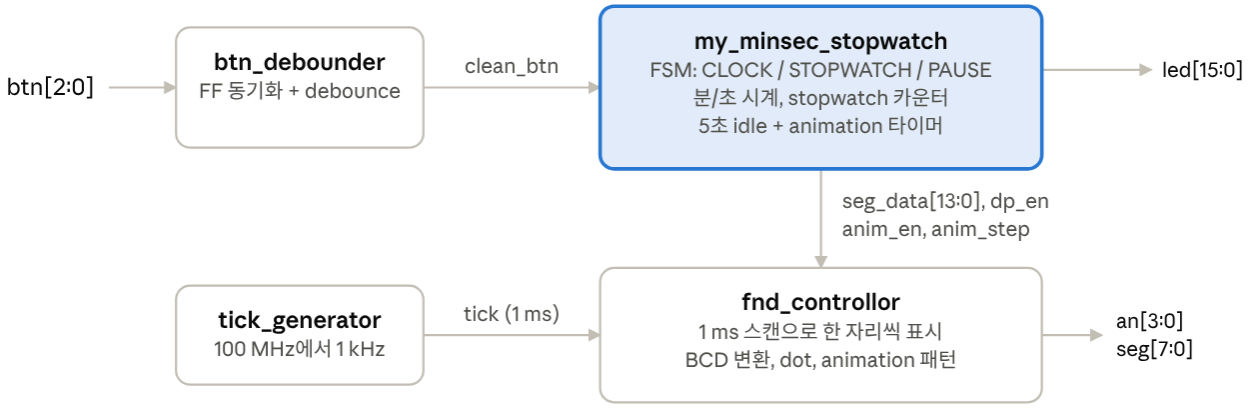
교수님이 추가로 요청한 기능 두 가지도 구현했다.

- **circle animation**: stopwatch가 일시정지된 상태에서 버튼을 누르지 않고 5초가 지나면 세그먼트 한 칸이 4자리 테두리를 따라 시계 방향으로 돈다. 버튼을 누르면 animation이 멈추고 해당 버튼의 기능을 수행한다.
- **분/초 사이 dot 깜박임**: 시계 모드에서 분과 초 사이의 점이 1초에 한 번 깜박인다(0.5초 켜짐, 0.5초 꺼짐).

2. 구현 내용

2.1 block diagram

버튼 입력은 debounce와 FSM을 거쳐 FND 스캔으로 표시된다



clk, reset: 모든 모듈에 공통으로 연결

block diagram · 5개 모듈

debouncer가 만든 clean_btn을 my_minsec_stopwatch가 받아 상태와 표시값을 정하고, fnd_controllor가 1 ms tick에 맞춰 자리를 바꿔 가며 출력한다. 버튼을 누른 순간 펄스를 만드는 부분(`w_press`)과 FSM, animation 타이머는 모두 my_minsec_stopwatch 안에 있다.

2.2 FSM

Left와 Right 두 버튼이 세 상태를 오간다



CLOCK에서 Center와 Right, PAUSE에서 Left는 입력을 무시한다
화살표 라벨은 버튼을 누른 순간(1 클럭 펄스)의 입력이다

FSM · 3개 상태

상태는 `CLOCK`, `STOPWATCH`, `PAUSE` 세 가지이고, 전원을 켜거나 reset하면 `CLOCK`에서 시작한다. 표시 모드와 LED는 현재 상태에 따라 정해진다. 분/초 시계는 상태와 관계없이 항상 동작하고, stopwatch 카운터는 `STOPWATCH` 상태에서만 증가한다. `PAUSE`에서 5초 동안 버튼을 누르지 않으면 상태는 그대로 두고 출력만 animation으로 바뀐다.

2.3 source code

프로젝트 소스는 6개이다. 교수님 제공 예제(09_up_down_counter)를 바탕으로 했고, 기존 `main_controller.v`는 사용하지 않아 프로젝트에서 빼고 `my_minsec_stopwatch.v`로 대체했다.

파일	역할	예제 대비
`top.v`	모듈 연결(최상위)	수정
`my_minsec_stopwatch.v`	FSM, 분/초 시계, stopwatch, animation 타이밍	신규
`fnd_controller.v`	FND 스캔, BCD 변환, dot과 animation 패턴 출력	수정
`tick_generator.v`	100 MHz에서 1 ms tick 생성	변경 없음
`btn_debounce10ms.v`	버튼 3개 2단 FF 동기화 + 10 ms debounce	수정
`tb_top.v`	시뮬레이션용 self-checking test bench	신규

top.v

```
`timescale 1ns / 1ps

module top(input clk, reset, [2:0]btn, [7:0]sw, output [7:0]seg, [3:0]an,
[15:0]led);

    wire [2:0]w_clean_btn;
    wire [13:0]w_seg_data;
    wire w_tick;
    wire w_dp_en;
    wire w_anim_en;
    wire [3:0] w_anim_step;
    btn_debounder u_btn_debounder(.clk(clk), .reset(reset), .noisy_btn(btn),
.clean_btn(w_clean_btn));

    my_minsec_stopwatch u_my_minsec_stopwatch(.clk(clk), .reset(reset),
.btn(w_clean_btn), .seg_data(w_seg_data), .dp_en(w_dp_en),
.anim_en(w_anim_en), .anim_step(w_anim_step), .led(led));

    tick_generator u_tick_generator(.clk(clk), .reset(reset), .tick(w_tick));

    fnd_controllor u_fnd_controllor(.clk(clk), .reset(reset), .tick(w_tick),
.dp_en(w_dp_en), .anim_en(w_anim_en), .anim_step(w_anim_step),
.in_data(w_seg_data), .an(an), .seg(seg));

endmodule
```

my_minsec_stopwatch.v

```
`timescale 1ns / 1ps

// 분/초 시계 <-> stopwatch(10ms) <-> 일시정지
//   btn[0] Left   : 분/초 시계 <-> stopwatch
//   btn[2] Right  : stopwatch <-> 일시정지
//   btn[1] Center : stopwatch 값 reset (0)
//   power reset   : 분/초 시계 상태로 시작
//   분/초 시계    : 분과 초 사이의 dot 가 1초에 1번 깜박임
//   일시정지      : 버튼 입력 없이 5초가 지나면 circle animation, 버튼을 누르면
animation 중지
module my_minsec_stopwatch #(
    parameter CNT_10MS = 1_000_000,      // 100MHz clk 기준 10ms
    parameter IDLE_10MS = 500,           // 일시정지에서 animation 시작까지 (10ms
x 500 = 5초)
    parameter ANIM_STEP_10MS = 10        // animation 한 칸 이동 시간 (10ms x 10
= 0.1초)
    )(
    input clk,
    input reset,
    input [2:0] btn,             // debounce 된 버튼 {Right, Center, Left}
    output [13:0] seg_data,      // FND 에 표시할 값 (10진수 4자리)
    output dp_en,                // 가운데 점 표시
    output anim_en,              // circle animation 동작 중
    output [3:0] anim_step,      // animation 위치 (0 ~ 11)
    output [15:0] led            // led[15:14] : 현재 상태
    );

    localparam CLOCK     = 2'b00;    // 분/초 시계    : mm.ss
    localparam STOPWATCH = 2'b01;    // stopwatch    : ss.cc (cc = 10ms 단위)
    localparam PAUSE     = 2'b10;    // 일시정지

    reg [1:0] r_state;
    reg [2:0] r_prev_btn;
    wire [2:0] w_press = btn & ~r_prev_btn;      // 버튼을 누른 순간 1clk 펄스

    // ---- 10ms tick, 1s tick (분/초 시계는 항상 동작)
    reg [$clog2(CNT_10MS)-1:0] r_cnt_10ms;
    reg [6:0] r_cnt_100;
    wire w_tick_10ms = (r_cnt_10ms == CNT_10MS - 1);
    wire w_tick_1s   = w_tick_10ms && (r_cnt_100 == 99);

    always @(posedge clk, posedge reset) begin
        if (reset) begin
            r_cnt_10ms <= 0;
```

```
            r_cnt_100 <= 0;
        end else begin
            if (w_tick_10ms) begin
                r_cnt_10ms <= 0;
                if (r_cnt_100 == 99) r_cnt_100 <= 0;
                else r_cnt_100 <= r_cnt_100 + 1;
            end else begin
                r_cnt_10ms <= r_cnt_10ms + 1;
            end
        end
    end

    // ---- 분/초 시계 : 00.00 ~ 59.59
    reg [5:0] r_sec, r_min;

    always @(posedge clk, posedge reset) begin
        if (reset) begin
            r_sec <= 0;
            r_min <= 0;
        end else if (w_tick_1s) begin
            if (r_sec == 59) begin
                r_sec <= 0;
                if (r_min == 59) r_min <= 0;
                else r_min <= r_min + 1;
            end else begin
                r_sec <= r_sec + 1;
            end
        end
    end

    // ---- 상태(state) 천이 + stopwatch counter : 00.00 ~ 99.99
    reg [13:0] r_sw_cnt;

    always @(posedge clk, posedge reset) begin
        if (reset) begin
            r_state <= CLOCK;
            r_prev_btn <= 3'b000;
            r_sw_cnt <= 0;
        end else begin
            r_prev_btn <= btn;

            // stopwatch 는 STOPWATCH 상태에서만 10ms 마다 증가
            if (r_state == STOPWATCH && w_tick_10ms) begin
                if (r_sw_cnt == 9999) r_sw_cnt <= 0;
                else r_sw_cnt <= r_sw_cnt + 1;
            end
```

```
            case (r_state)
                CLOCK: begin
                    if (w_press[0]) r_state <= STOPWATCH;       // Left
                end
                STOPWATCH: begin
                    if (w_press[0]) r_state <= CLOCK;           // Left
                    else if (w_press[2]) r_state <= PAUSE;      // Right
                    if (w_press[1]) r_sw_cnt <= 0;              // Center :
reset
                end
                PAUSE: begin
                    if (w_press[2]) r_state <= STOPWATCH;       // Right
                    if (w_press[1]) r_sw_cnt <= 0;              // Center :
reset
                end
                default: r_state <= CLOCK;
            endcase
        end
    end

    // ---- 일시정지에서 버튼 입력이 없는 시간을 재서, 5초가 지나면 circle animation
    wire w_any_press = w_press[0] | w_press[1] | w_press[2];     // 버튼 3개 중
하나라도 눌렀으면 1
    reg [$clog2(IDLE_10MS+1)-1:0] r_idle_cnt;     // 버튼 입력 없는 시간 (10ms 단
위)
    reg [3:0] r_anim_cnt;
    reg [3:0] r_anim_step;

    always @(posedge clk, posedge reset) begin
        if (reset) begin
            r_idle_cnt <= 0;
            r_anim_cnt <= 0;
            r_anim_step <= 0;
        end else begin
            if (r_state != PAUSE || w_any_press) begin      // 버튼을 누르면
animation 중지
                r_idle_cnt <= 0;
                r_anim_cnt <= 0;
                r_anim_step <= 0;
            end else if (w_tick_10ms) begin
                if (r_idle_cnt < IDLE_10MS) begin
                    r_idle_cnt <= r_idle_cnt + 1;
                end else begin                              // 5초 경과 :
animation 동작
                    if (r_anim_cnt == ANIM_STEP_10MS - 1) begin
```

```
                        r_anim_cnt <= 0;
                        if (r_anim_step == 11) r_anim_step <= 0;
                        else r_anim_step <= r_anim_step + 1;
                    end else begin
                        r_anim_cnt <= r_anim_cnt + 1;
                    end
                end
            end
        end
    end

    assign anim_en = (r_state == PAUSE) && (r_idle_cnt >= IDLE_10MS);
    assign anim_step = r_anim_step;

    assign seg_data = (r_state == CLOCK) ? (r_min * 100 + r_sec) : r_sw_cnt;
    assign dp_en = (r_state == CLOCK) ? (r_cnt_100 < 50) : 1'b1;     // 시계 :
1초에 1번 깜박 (0.5초 켜짐, 0.5초 꺼짐)
    assign led = {r_state, 14'b0};
endmodule
```

fnd_controller.v

예제에서 `dp_en`, `anim_en`, `anim_step` 입력과 `r_anim_seg` 블록, 그리고 seg 출력의 마지막 두 줄(dot, animation)이 추가되었다. 나머지는 예제 그대로이다.

```
`timescale 1ns / 1ps
module fnd_controllor(input clk, reset, tick, dp_en, anim_en, [3:0]anim_step,
[13:0]in_data, output [3:0]an, [7:0]seg);

    wire [1:0] w_sel;
    fnd_digit_select u_fnd_digit_select(.clk(clk), .reset(reset),
.tick(tick), .sel(w_sel));

    wire [3:0]w_d1, w_d10, w_d100, w_d1000;
    bin2bcd4_digit u_bin2bcd4_digit(.in_data(in_data), .d1(w_d1),
.d10(w_d10), .d100(w_d100), .d1000(w_d1000));

    fnd_digit_display u_fnd_digit_display(.dp_en(dp_en), .anim_en(anim_en),
.anim_step(anim_step), .sel(w_sel), .d1(w_d1), .d10(w_d10), .d100(w_d100),
.d1000(w_d1000), .an(an), .seg(seg));
endmodule

// logic select 1_digit displaying fnd every 1ms,
module fnd_digit_select(input clk, reset, tick, output reg[1:0] sel);

    always @(posedge reset, posedge tick) begin
```

```
        if(reset) begin
            sel <= 0;
        end else begin
            if(tick) begin
                sel <= sel + 1;
            end
        end
    end
endmodule

// in_data : bin size express to 9999 in 14bit fnd
// 0000 ~ 9999 /digit number 0 ~ 9 bcd 4bit express
module bin2bcd4_digit(input [13:0] in_data, output [3:0]d1, [3:0]d10,
[3:0]d100, [3:0]d1000);

    assign d1 = in_data % 10;
    assign d10 = (in_data / 10) % 10;
    assign d100 = (in_data / 100) % 10;
    assign d1000 = (in_data / 1000) % 10;
endmodule

module fnd_digit_display(input dp_en, anim_en, [3:0]anim_step, [1:0]sel,
[3:0]d1, [3:0]d10, [3:0]d100, [3:0]d1000, output reg [3:0]an, reg [7:0]seg);

    reg [3:0] bcd_data;
    reg [7:0] r_anim_seg;

    // circle animation : 4자리 바깥 테두리를 따라 segment 한 칸이 시계방향으로 돈다
    //  0~3  : 위쪽 a (왼쪽 -> 오른쪽)   4,5 : 오른쪽 끝 b, c   6~9 : 아래쪽 d (오른
쪽 -> 왼쪽)   10,11 : 왼쪽 끝 e, f
    always @(*) begin
        r_anim_seg = 8'b11111111;                                       // 모두 꺼
짐
        case (anim_step)
            4'd0:  if (sel == 2'b11) r_anim_seg = 8'b11111110;      // a
            4'd1:  if (sel == 2'b10) r_anim_seg = 8'b11111110;
            4'd2:  if (sel == 2'b01) r_anim_seg = 8'b11111110;
            4'd3:  if (sel == 2'b00) r_anim_seg = 8'b11111110;
            4'd4:  if (sel == 2'b00) r_anim_seg = 8'b11111101;      // b
            4'd5:  if (sel == 2'b00) r_anim_seg = 8'b11111011;      // c
            4'd6:  if (sel == 2'b00) r_anim_seg = 8'b11110111;      // d
            4'd7:  if (sel == 2'b01) r_anim_seg = 8'b11110111;
            4'd8:  if (sel == 2'b10) r_anim_seg = 8'b11110111;
            4'd9:  if (sel == 2'b11) r_anim_seg = 8'b11110111;
            4'd10: if (sel == 2'b11) r_anim_seg = 8'b11101111;      // e
            4'd11: if (sel == 2'b11) r_anim_seg = 8'b11011111;      // f
```

```
            default: r_anim_seg = 8'b11111111;
        endcase
    end

    always@(sel) begin
        case(sel)
            2'b00: begin
                bcd_data = d1;
                an = 4'b1110;
            end
            2'b01: begin
                bcd_data = d10;
                an = 4'b1101;
            end
            2'b10: begin
                bcd_data = d100;
                an = 4'b1011;
            end
            2'b11: begin
                bcd_data = d1000;
                an = 4'b0111;
            end
            default: begin
                bcd_data = 4'b0000;
                an = 4'b1111;
            end
        endcase
    end

    always @(*) begin
        case(bcd_data)
            4'd0: seg = 8'b11000000;   // 0
            4'd1: seg = 8'b11111001;   // 1
            4'd2: seg = 8'b10100100;   // 2
            4'd3: seg = 8'b10110000;   // 3
            4'd4: seg = 8'b10011001;   // 4
            4'd5: seg = 8'b10010010;   // 5
            4'd6: seg = 8'b10000010;   // 6
            4'd7: seg = 8'b11111000;   // 7
            4'd8: seg = 8'b10000000;   // 8
            4'd9: seg = 8'b10010000;   // 9
            default: seg = 8'b11111111;  // all off
        endcase
        if (dp_en && sel == 2'b10) seg[7] = 1'b0;   // 가운데 점(dp) : 3번째 자
리(an[2])에 표시
```

```
        if (anim_en) seg = r_anim_seg;              // animation 중에는
animation 패턴 출력
    end
endmodule
```

tick_generator.v (변경 없음)

100 MHz 클럭을 100,000번 세어 1 ms마다 1 클럭 펄스를 만든다. FND가 자리를 바꾸는 속도(1 kHz)가 이것으로 결정된다.

```
`timescale 1ns / 1ps

module tick_generator(input clk, input reset, output reg tick);
    parameter INPUT_FREQ = 100_000_000;
    parameter TICK_Hz = 1000;
    parameter TICK_COUNT = INPUT_FREQ / TICK_Hz;

    reg[$clog2(TICK_COUNT)-1:0] r_tick_counter = 0;

    always @(posedge clk or posedge reset) begin
        if(reset) begin
            tick <= 0; r_tick_counter <= 0;
        end else begin
            if(r_tick_counter == TICK_COUNT - 1) begin
                r_tick_counter <= 0;
                tick <= 1'b1;
            end else begin
                r_tick_counter <= r_tick_counter + 1;
                tick <= 1'b0;
            end
        end
    end
endmodule
```

btn_debounce10ms.v (2단 FF 동기화 추가)

버튼 입력을 2단 FF(`r_FF1`, `r_FF2`)로 클럭에 동기화한 뒤, 값이 바뀔 후 10 ms(999,999 클럭) 동안 유지될 때에만 clean_btn을 바꾼다. 비동기 버튼 입력 때문에 생기는 metastability를 줄이고, 채터링은 10 ms보다 짧아 걸러진다. 예제의 `debouncer`에서 바뀐 부분은 `r_FF1`, `r_FF2`를 추가하고 `noisy_btn` 대신 `r_FF2`를 쓰도록 한 것뿐이다.

```
module btn_debounder(
    input      clk,
    input      reset,
    input      [2:0]noisy_btn,  // raw noisy button input
    output     [2:0]clean_btn
```

```
);
    debouncer u_btnL_debounder(
        .clk(clk), .reset(reset), .noisy_btn(noisy_btn[0]),
.clean_btn(clean_btn[0])
    );
    debouncer u_btnC_debounder(
        .clk(clk), .reset(reset), .noisy_btn(noisy_btn[1]),
.clean_btn(clean_btn[1])
    );
    debouncer u_btnR_debounder(
        .clk(clk), .reset(reset), .noisy_btn(noisy_btn[2]),
.clean_btn(clean_btn[2])
    );
endmodule

module debouncer #(parameter DEBOUNCE_LIMIT = 20'd999_999) (
    input      clk,
    input      reset,
    input      noisy_btn,  // raw noisy button input
    output reg clean_btn
);
    reg [19:0] r_count;
    reg r_btn_state=0;
    reg r_FF1, r_FF2;   // 2단 플립플롭 동기화 (비동기 버튼 입력을 clk 에 동기화)

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_FF1 <= 0;
            r_FF2 <= 0;
        end else begin
            r_FF1 <= noisy_btn;
            r_FF2 <= r_FF1;
        end
    end

    always @(posedge clk or posedge reset) begin
        if (reset) begin   // active-high reset
            r_count <= 0;
            r_btn_state <= 0;
            clean_btn <= 0;
        end else if (r_FF2 == r_btn_state) begin  // 버튼 상태가 이전과 동일할 경우
(안정됨)
            r_count <= 0;
        end else begin
            if (r_count < DEBOUNCE_LIMIT)  // 버튼 상태가 바뀌었지만 아직 안정되지 않
은 경우
```

```
                r_count <= r_count + 1;
            else begin  // 상태가 충분히 오랫동안 유지됨(10ms)
                r_btn_state <= r_FF2;
                clean_btn <= r_FF2;
                r_count <= 0;  // clear하면 다음 변경을 다시 감지할 수 있음
            end
        end
    end
endmodule
```

2.4 test bench 작성 / timing chart 분석

test bench 구성. `tb_top.v`는 `top`을 그대로 불러서 버튼 입력만 넣고, 결과를 스스로 검사하는 (self-checking) 방식으로 작성했다. 마지막에 `ALL PASS` 또는 `ERRORS = n`을 출력한다.

- **시간 단축**: 실제 시간으로는 시뮬레이션이 너무 길어 `defparam`으로 `CNT_10MS`를 10으로 줄였다. 10 ms가 100 ns가 되어 1초가 10 µs로 줄어든다. `DEBOUNCE_LIMIT`도 9(100 ns)로 줄였다. `IDLE_10MS`(5초 = 50 µs)는 그대로 둔다.
- **채터링 모사**: `press` task는 버튼을 37, 23, 41, 19 ns 간격으로 여러 번 흔든 뒤 500 ns 동안 안정적으로 누른다. debouncer가 짧은 흔들림을 걸러내는지 함께 확인한다.
- **자동 검사**: `chk` task가 조건을 확인해 `OK`나 `FAIL`을 찍고, 틀리면 `r_err`를 1 올린다.
- **FND tick**: 자리 바꿈 tick(1 ms)은 줄이지 않았다. 그래서 파형에서 `an`, `seg`는 1000 µs에 한 번만 바뀐다. 이는 시뮬레이션 설정 때문이며 보드에서는 1 ms마다 자리가 바뀐다.

결과. 검사 26개가 모두 `OK`였고 `r_err`는 끝까지 0이었다. 시뮬레이션은 1,860,030 ns에서 `$finish`로 끝났다.

timing chart 분석. 시뮬레이션에서 1초는 10 µs이다. 아래는 파형을 시간 순서대로 읽은 결과이다.

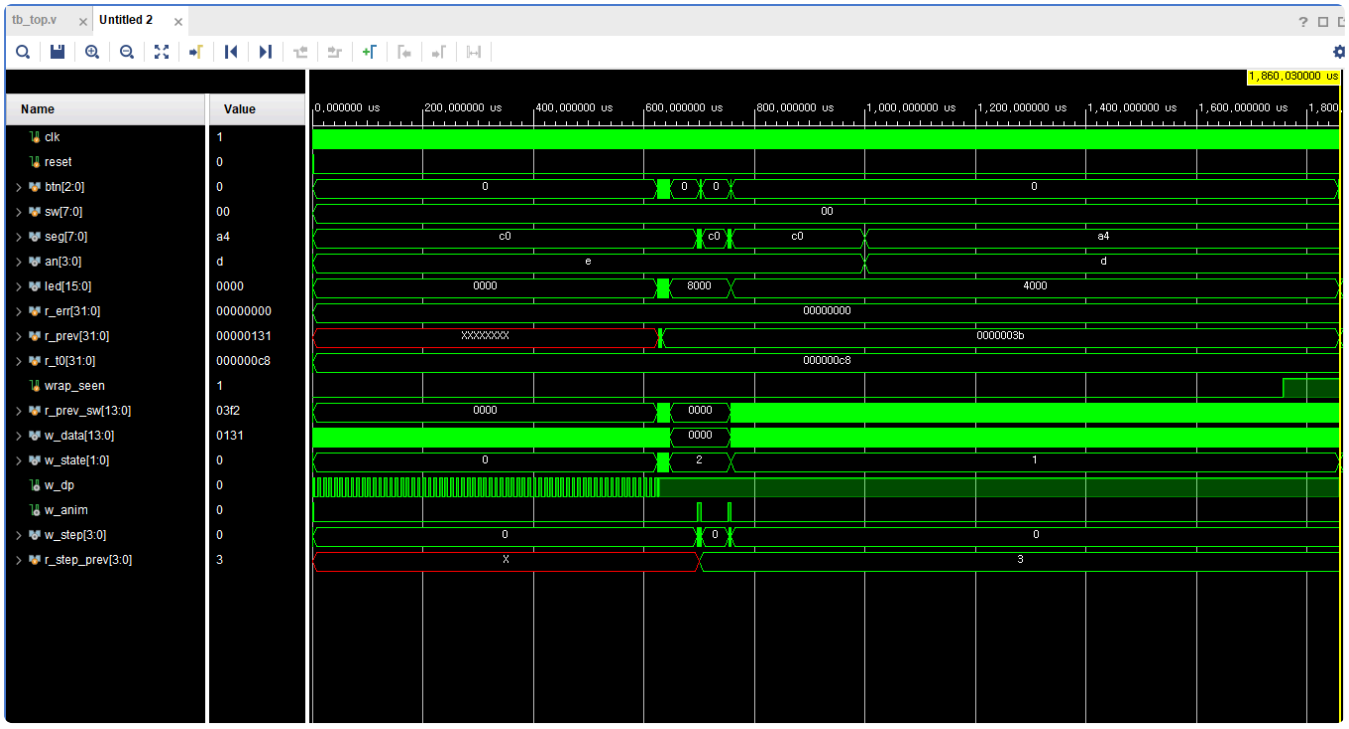


그림. tb_top 시뮬레이션 파형. 가로축 한 칸이 200 µs이며, 시뮬레이션에서 1초는 10 µs이다.

시간(대략)	파형	의미
0 ~ 620 µs	`w_state`=0, `led`=0000, `w_dp`가 5 µs마다 토글	분/초 시계 모드. dot이 0.5초 켜지고 0.5초 꺼진다.
625 ~ 647 µs	`btn`과 `w_state`가 빠르게 바뀜	Left, Right, Left(무시), Right, Center, Right, Center 순으로 누르는 구간
647 ~ 758 µs	`w_state`=2, `led`=8000, `w_data`=0000, `w_dp` 고정 1	일시정지. Center로 0이 된 값이 유지된다.
약 696 µs, 약 751 µs	`w_anim`이 짧게 올라감	마지막 버튼 입력 뒤 50 µs(5초)가 지나 animation이 시작되고, 버튼을 누르면 멈춘다.
758 ~ 1858 µs	`w_state`=1, `led`=4000, `w_data` 증가	stopwatch가 0부터 다시 증가한다.
약 1758 µs	`wrap_seen`이 1이 됨	99.99에서 00.00으로 넘어간다. (9999 × 100 ns ≈ 1000 µs 뒤)
1860 µs	`w_state`=0, `w_data`=0131	Left로 시계 모드에 돌아왔고 0131은 십진수 305, 즉 3분 05초이다.

세 가지를 추가로 확인했다.

- `r_step_prev`가 3으로 잡힌다. animation이 시작된 지 0.3초 뒤라서 step 3이다.
- `led[15:14]`는 시계 00, stopwatch 01, 일시정지 10으로 상태를 따라간다.

- stopwatch를 멈춘 값은 59(0.59초)이고, 일시정지 동안 변하지 않고 Right로 재개하면 그 값에서 이어진다.

검사 항목 (26개).

구분	항목
시계	reset 후 00.00, dot 켜짐/꺼짐/켜짐(0.25, 0.75, 1.25초), 3초, 분 올림(01.02)
상태 전환	Left로 stopwatch, Right로 일시정지, 일시정지에서 Left 무시, Right로 재개, Left로 시계 복귀, 시계에서 Center 무시
stopwatch	0.5초 증가량, 일시정지 중 값 유지, Center로 0부터 다시 시작, 99.99 → 00.00
animation	5초 전에는 없음, 5초 후 시작, step 이동, 버튼으로 중지, 다시 5초 후 재시작, Right로 중지와 재개
시계 유지	stopwatch 동안에도 시계가 계속 감

tb_top.v

```
`timescale 1ns / 1ps

module tb_top();
    reg clk = 0;
    reg reset = 0;
    reg [2:0] btn = 3'b000;     // btn[0] Left, btn[1] Center, btn[2] Right
    reg [7:0] sw = 8'b0;
    wire [7:0] seg;
    wire [3:0] an;
    wire [15:0] led;

    integer r_err = 0;
    integer r_prev;
    integer r_t0;
    reg wrap_seen = 0;
    reg [13:0] r_prev_sw = 0;

    top dut(.clk(clk), .reset(reset), .btn(btn), .sw(sw), .seg(seg), .an(an),
.led(led));

    // 시뮬레이션 속도를 위해 시간을 줄임 : 10ms -> 10clk(100ns), 1s -> 100 x 10ms =
10us, debounce 10clk
    defparam dut.u_my_minsec_stopwatch.CNT_10MS = 10;
```

```
    // IDLE_10MS(5초 = 500 x 10ms), ANIM_STEP_10MS(0.1초 = 10 x 10ms) 는 그대로
사용
    defparam dut.u_btn_debounder.u_btnL_debounder.DEBOUNCE_LIMIT = 9;
    defparam dut.u_btn_debounder.u_btnC_debounder.DEBOUNCE_LIMIT = 9;
    defparam dut.u_btn_debounder.u_btnR_debounder.DEBOUNCE_LIMIT = 9;

    always #5 clk = ~clk;   // 100MHz

    wire [13:0] w_data  = dut.w_seg_data;

    wire [1:0]  w_state = dut.u_my_minsec_stopwatch.r_state;   // 0:시계
1:stopwatch 2:pause
    wire        w_dp    = dut.w_dp_en;
    wire        w_anim  = dut.w_anim_en;
    wire [3:0]  w_step  = dut.w_anim_step;
    reg  [3:0]  r_step_prev;

    // stopwatch 가 9999 -> 0 으로 넘어가는 순간 확인
    always @(posedge clk) begin
        r_prev_sw <= dut.u_my_minsec_stopwatch.r_sw_cnt;
        if (r_prev_sw == 9999 && dut.u_my_minsec_stopwatch.r_sw_cnt == 0 &&
w_state == 1) wrap_seen <= 1;
    end

    // 버튼 i 를 눌렀다 뗌 (채터링 포함)
    task press(input integer i);
    begin
        btn[i] = 1; #37; btn[i] = 0; #23; btn[i] = 1; #41; btn[i] = 0; #19;
btn[i] = 1;
        #500;
        btn[i] = 0; #29; btn[i] = 1; #31; btn[i] = 0;
        #300;
    end
    endtask

    task chk(input [8*120-1:0] name, input cond);
    begin
        if (cond) $display("OK   %0s : data=%0d state=%0d", name, w_data,
w_state);
        else begin r_err = r_err + 1; $display("FAIL %0s : data=%0d
state=%0d", name, w_data, w_state); end
    end
    endtask

    // mm.ss 로 표시된 값 -> 초
    function integer to_sec(input [13:0] d);
```

```
        to_sec = (d / 100) * 60 + (d % 100);
    endfunction

    initial begin
        #100; reset = 1; #100; reset = 0; r_t0 = $time;
        #50;
        chk("power reset -> clock mode 00.00", w_state == 0 && w_data == 0);

        // 분/초 시계 : dot 가 1초에 1번 깜박 (0.5초 켜짐 / 0.5초 꺼짐)
        #2_000;  chk("clock dot on  (0.25 sec)", w_dp == 1);
        #5_000;  chk("clock dot off (0.75 sec)", w_dp == 0);
        #5_000;  chk("clock dot on  (1.25 sec)", w_dp == 1);
        #23_000;                                    // 여기까지 합쳐서 3.5 초
        chk("clock 3 sec", w_data == 3);

        #(59*10_000);                               // 약 62.5 초
        chk("clock minute carry 01.02", w_data == 102);

        // Left : 시계 -> stopwatch
        press(0);
        chk("Left -> stopwatch", w_state == 1);
        r_prev = w_data;
        #5_000;                                     // 50 x 10ms
        chk("stopwatch counting (about 0.50 sec)", w_data > r_prev + 45 &&
w_data < r_prev + 55);

        // Right : stopwatch -> pause
        press(2);
        chk("Right -> pause", w_state == 2);
        r_prev = w_data;
        #5_000;
        chk("pause holds value", w_data == r_prev);

        // Left 는 pause 에서 무시
        press(0);
        chk("Left ignored in pause", w_state == 2 && w_data == r_prev);

        // Right : pause -> stopwatch (이어서 증가)
        press(2);
        chk("Right -> stopwatch resume", w_state == 1);
        #5_000;
        chk("counting after resume", w_data > r_prev + 45);

        // Center : stopwatch reset (동작 중)
        press(1);
        chk("Center -> restart from 0", w_state == 1 && w_data < 10);
```

```
        // pause 상태에서 Center
        press(2);
        press(1);
        chk("Center in pause -> 0, stay pause", w_state == 2 && w_data == 0);

        // 일시정지에서 버튼 입력 없이 5초(= 50us) 가 지나면 circle animation
        #45_000;                                    // 약 4.6초
        chk("no animation before 5 sec", w_anim == 0);
        #8_000;                                     // 약 5.4초
        chk("animation starts after 5 sec", w_anim == 1 && w_state == 2);
        r_step_prev = w_step;
        #3_000;
        chk("animation step moves", w_anim == 1 && w_step != r_step_prev);

        // 버튼을 누르면 animation 중지 (pause 에서 Left 는 기능 없음 -> 상태 유지)
        press(0);
        chk("button stops animation (Left)", w_anim == 0 && w_state == 2);
        #45_000;
        chk("no animation again before 5 sec", w_anim == 0);
        #8_000;
        chk("animation restarts after 5 sec", w_anim == 1);

        // 동작 버튼(Right)을 누르면 animation 중지 + 해당 기능 수행 (stopwatch 재개)
        press(2);
        chk("Right stops animation and resumes stopwatch", w_anim == 0 &&
w_state == 1);

        // stopwatch 99.99 -> 00.00 wrap (9999 x 100ns 약 1ms)
        wrap_seen = 0;
        #1_100_000;
        chk("stopwatch wraps 99.99 -> 00.00", wrap_seen);

        // Left : stopwatch -> 시계, 시계는 계속 동작했음
        press(0);
        chk("Left -> clock mode", w_state == 0);
        chk("clock kept running in background", to_sec(w_data) >= to_sec(102)
+ 1 && to_sec(w_data) <= (($time - r_t0) / 10_000) + 1);

        // Center 는 시계 상태에서 무시
        r_prev = w_data;
        press(1);
        chk("Center ignored in clock mode", w_state == 0 && w_data >=
r_prev);

        if (r_err == 0) $display("ALL PASS");
```

```
        else $display("ERRORS = %0d", r_err);
        $finish;
    end
endmodule
```

2.5 동작 동영상

Basys3 보드에서 실제로 동작하는 모습을 촬영해 첨부한다. 아래 순서로 촬영하면 요구사항을 빠짐없이 보여 줄 수 있다. 시작 전에 SW15(reset)는 내려 둔다.

1. 전원을 켜서 분/초 시계가 `00.00` 부터 올라가고 가운데 점이 1초에 한 번 깜박이는 것을 보여 준다.
2. Left를 눌러 stopwatch를 시작하고 `led[14]` 가 켜지는 것을 보여 준다.
3. Right로 일시정지하고(`led[15]` 켜짐) 값이 멈추는 것을 보여 준다.
4. 버튼을 누르지 않고 5초를 기다려 circle animation이 시작되는 것을 보여 준다.
5. 버튼을 눌러 animation이 멈추고 다시 숫자가 보이는 것, Right로 재개되는 것을 보여 준다.
6. Center로 `00.00` 초기화를 보여 주고, Left로 시계 모드에 돌아와 시계가 계속 갔던 것을 보여 준다.

동영상: (링크 또는 파일을 여기에 첨부)

3. trouble shooting 및 구현 후기

3.1 trouble shooting

문제	원인	해결
합성 시 `valid license was not found ... device 'xcvc1802'` 오류	프로젝트 타깃 칩이 Basys3의 FPGA가 아니라 Versal 계열 xcvc1802로 잡혀 있었다. 무료(WebPACK) 라이선스에 포함되지 않는 칩이다.	Settings > General > Project device에서 칩을 `xc7a35tcpg236-1` 로 바꾸었다.
Bitstream 생성 실패 (1): `reset` 필 미지정	XDC의 `reset` (SW15) 줄이 주석 처리되어 있어 포트에 핀이 없었다(UCIO/NSTD 오류).	주석을 풀고 포트명을 `reset` 으로 맞추었다.
Bitstream 생성 실패 (2): XDC 문법 오류	XDC는 Tcl이다. `btn[0]` 처럼 인덱스가 있는 포트를 중괄호 없이 썼고, 줄 끝 주석에 `//` 나 `;` 없는 `#` 을 썼다.	`{btn[0]}` 처럼 중괄호로 감싸고, 줄 끝 주석은 `;#` 로 썼다.
버튼 방향이 반대로 동작	XDC에서 Left(W19)와 Center(U18)의 `btn` 번호가 바뀌어 있었다.	Left는 `btn[0]` (W19), Center는 `btn[1]` (U18), Right는 `btn[2]` (T17)로 맞추었다.

문제	원인	해결
파형에서 `an`, `seg`가 1000 µs에 한 번만 바뀜	시뮬레이션 속도를 위해 10 ms와 debounce만 줄였고, FND 자리 바꿈 tick(1 ms)은 그대로였다.	버그가 아니라 설정 때문임을 확인했다. 보드에서는 1 ms마다 자리가 바뀌어 문제가 없다.

3.2 구현 후기

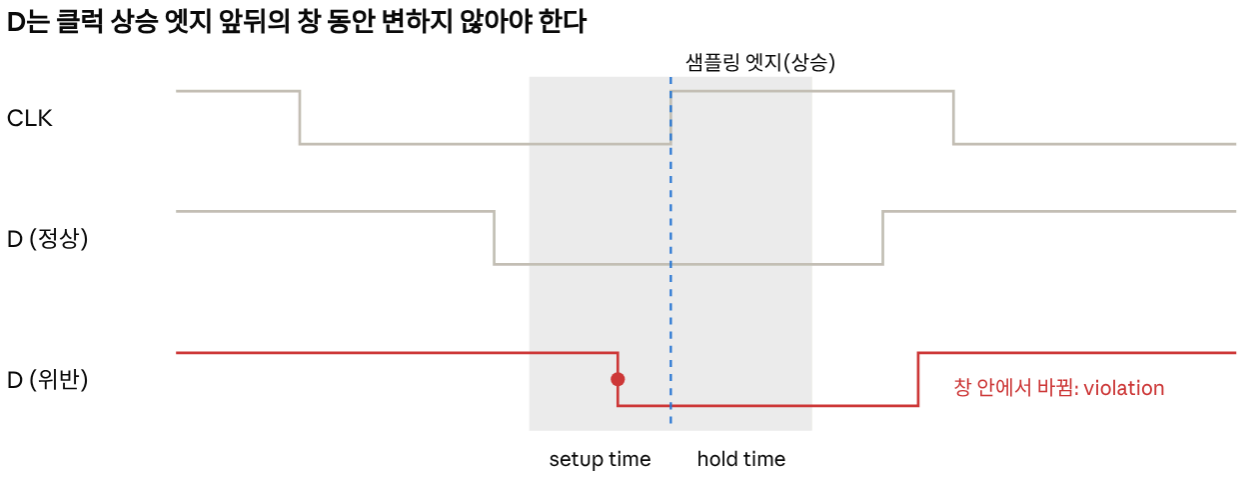
- 버튼을 누른 순간을 1 클럭 펄스(`btn & ~r_prev_btn`)로 만들어 쓰면 버튼 한 번에 상태가 여러 번 바뀌는 문제를 막을 수 있다는 것을 배웠다.
- 시뮬레이션에서는 `defparam`으로 시간 파라미터를 줄이고 self-checking test bench로 검사하면 긴 동작도 짧은 시뮬레이션으로 확인할 수 있다.
- 보드 업로드 전에 칩 설정과 XDC의 포트명, 핀 번호, 문법을 한 번 더 확인해야 시간을 아낄 수 있다는 것을 알게 되었다.
- 한 모듈에 기능을 모두 넣기보다 시계, stopwatch, FSM, animation 타이밍을 나눈 뒤 `my_minsec_stopwatch` 하나에서 조합하니 수정하기 쉽았다.

4. Background 기술 Summary

4.1 setup / hold time

개념. flip-flop(FF)은 클럭의 상승 엣지에서 D 입력을 샘플링해 Q로 내보낸다. 이때 입력은 엣지 앞뒤의 일정 시간 동안 안정적이어야 한다.

- **setup time (t_su)**: 클럭 엣지 *전에* 데이터가 미리 안정되어 있어야 하는 최소 시간
- **hold time (t_h)**: 클럭 엣지 *뒤에* 데이터가 바뀌지 않고 유지되어야 하는 최소 시간



setup/hold 창 · D 파형 2가지

회색 영역이 클럭 엣지 앞뒤의 setup/hold 창이다. D(정상)는 창 안에서 바뀌지 않아 안전하고, D(위반)은 setup 창 안에서 바뀌어 샘플링 값이 불확실하다.

원인과 문제점. 두 시간을 지키지 못하는 경우를 violation이라 하며, 원인은 서로 다르다.

구분	원인	문제점
setup violation	클럭 주기가 너무 짧거나, FF 사이 조합 회로 지연이 너무 큼	새 데이터가 제때 도착하지 못해 이전 값이나 잘못된 값이 저장된다.
hold violation	데이터 경로가 너무 짧거나 클럭 skew가 커서 데이터가 엣지 직후에 바뀜	엣지에서 잡아야 할 값이 다음 값으로 덮어써진다.
비동기 입력	버튼이나 스위치처럼 클럭과 무관하게 바뀌는 신호가 샘플링 창 안에서 바뀜	두 시간을 보장할 수 없어 출력이 불안정해진다(4.2 metastability).

setup을 만족하는 조건은 다음 식으로 표현한다. 우변의 값들의 합이 클럭 주기보다 작아야 한다.

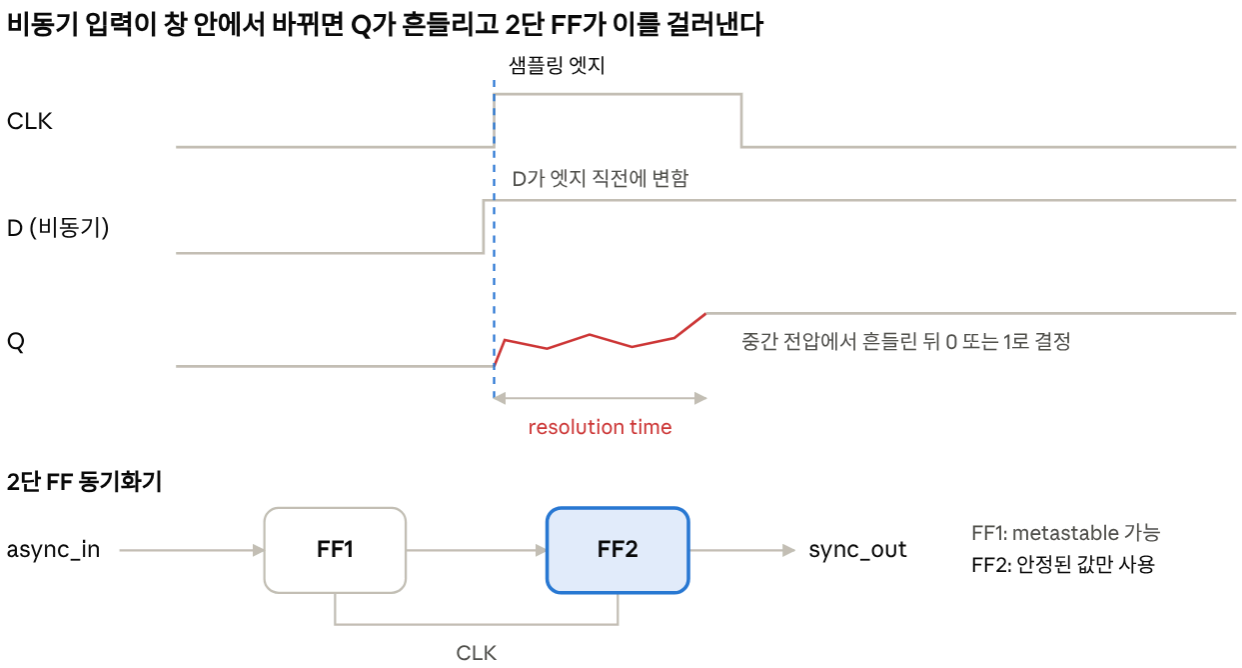
```
T_{clk} \ge T_{cq} + T_{logic} + T_{su} + T_{skew}
```

해결 방법.

- **setup violation**: 클럭 주파수를 낮추거나, 긴 조합회로를 중간에 FF를 두어 나눠(pipeline) 경로 지연을 줄인다.
- **hold violation**: 데이터 경로에 지연을 넣거나 클럭 skew를 줄인다. FPGA 툴이 배치배선 단계에서 자동으로 보정한다.
- **비동기 입력**: 2단 FF 동기화기를 거쳐 클럭에 맞춘 뒤 사용한다.
- 타이밍 문제는 XDC에 `create_clock`을 정확히 지정하고 Vivado의 timing report로 확인한다. 이 프로젝트는 `create_clock -period 10.00`(100 MHz)으로 제약했다.

4.2 Metastability

정의. FF의 setup/hold 시간을 어겨 입력이 샘플링 순간에 바뀌면, 출력 Q가 0도 1도 아닌 중간 전압에 머물러 있다가 일정 시간(resolution time)이 지난 뒤 임의의 0 또는 1로 결정되는 상태이다. 어느 쪽으로 결정될지, 얼마나 걸릴지 예측할 수 없다.



metastability 파형과 2단 FF 동기화기

D가 샘플링 엣지 직전에 바뀌면 Q는 resolution time 동안 중간 전압에서 흔들린다. 동기화기는 FF1이 흔들려도 FF2가 한 클럭 뒤에 안정된 값만 받게 한다.

원인.

- 클럭과 무관하게 바뀌는 비동기 입력(버튼, 스위치)이 샘플링 창 안에서 변한다.
- 서로 다른 클럭 도메인 사이의 신호를 동기화 없이 연결한다.

문제점. 불확실한 값이 다음 FF와 논리회로로 전파되면 서로 다른 논리가 같은 신호를 다르게 해석하거나 FSM이 정의되지 않은 상태로 가서 오동작한다. 한번 발생하면 재현이 어려워 디버깅도 어렵다.

해결 방법.

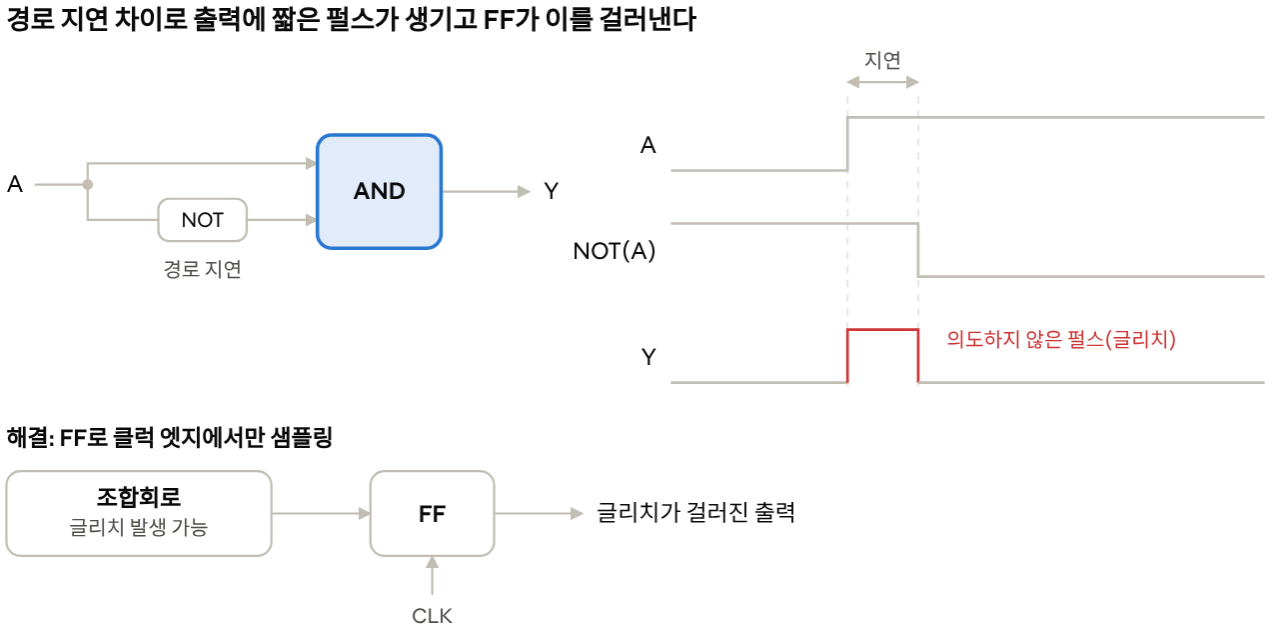
- **2단 FF 동기화기**: 비동기 입력을 FF 2개를 연속으로 거쳐 받는다. 첫 FF가 metastable이 되어도 한 클럭 주기 동안 안정될 시간을 주고, 두 번째 FF의 출력만 사용한다.
- 신뢰도가 더 필요하면 FF를 3단으로 늘리거나 클럭을 낮춰 resolution time을 확보한다.
- metastability는 없애는 것이 아니라 발생 확률을 줄이는 것이다. 동기화기의 신뢰도는 MTBF로 표현하며, 대기 시간 t_r이 길수록 지수적으로 늘어난다.

```
MTBF = \frac{e^{t_r / \tau}}{T_0 \cdot f_{clk} \cdot f_{data}}
```

이 프로젝트에서. `debouncer`에 2단 FF 동기화기(`r_FF1`, `r_FF2`)를 추가했다. 버튼 입력(`noisy_btn`)은 FF 2개를 거친 `r_FF2`만 debounce 로직에서 쓴다. 그 뒤 입력이 10 ms 동안 유지되는지 확인해 채터링을 걸러낸다. 동기화 때문에 입력이 2 클럭(20 ns) 늦어지지만 10 ms debounce에 비하면 무시할 수 있다. 같은 test bench로 다시 시뮬레이션해 `ALL PASS`를 확인했다.

4.3 Glitch

정의. 조합회로의 출력이 최종값으로 안정되기 전에, 짧은 시간 동안 의도하지 않은 값이 나타나는 짧은 펄스이다. 논리식으로는 출력이 바뀌지 않아야 하는데 한 번 튀는 경우(static hazard)와 입력 변화에 따라 중간값을 거치는 경우가 있다.



glitch 발생과 FF로 걸러내기

A가 0에서 1로 바뀔 때 NOT 경로가 느려 두 입력이 잠깐 모두 1이 되므로, 논리식으로는 항상 0이어야 하는 Y에 펄스가 생긴다.

원인.

- 입력 신호들이 게이트를 거쳐 도착하는 경로 지연이 서로 다르다(race).
- 예: 같은 신호 A가 직접 경로와 NOT을 거친 지연 경로로 나뉘어 같은 게이트에 들어가면, A가 바뀔 때 두 경로의 값이 잠깐 같이 1이 되어 출력에 펄스가 생긴다.
- 여러 비트가 동시에 바뀌는 신호(예: 카운터 01에서 10)는 중간에 00이나 11을 거칠 수 있다.

문제점. 시스템이 글리치를 유효한 값으로 읽으면 오동작한다. 특히 조합회로 출력이 클럭이나 비동기 reset, enable로 쓰이면 글리치가 엣지로 인식되어 카운터가 엉뚱하게 증가하거나 reset된다.

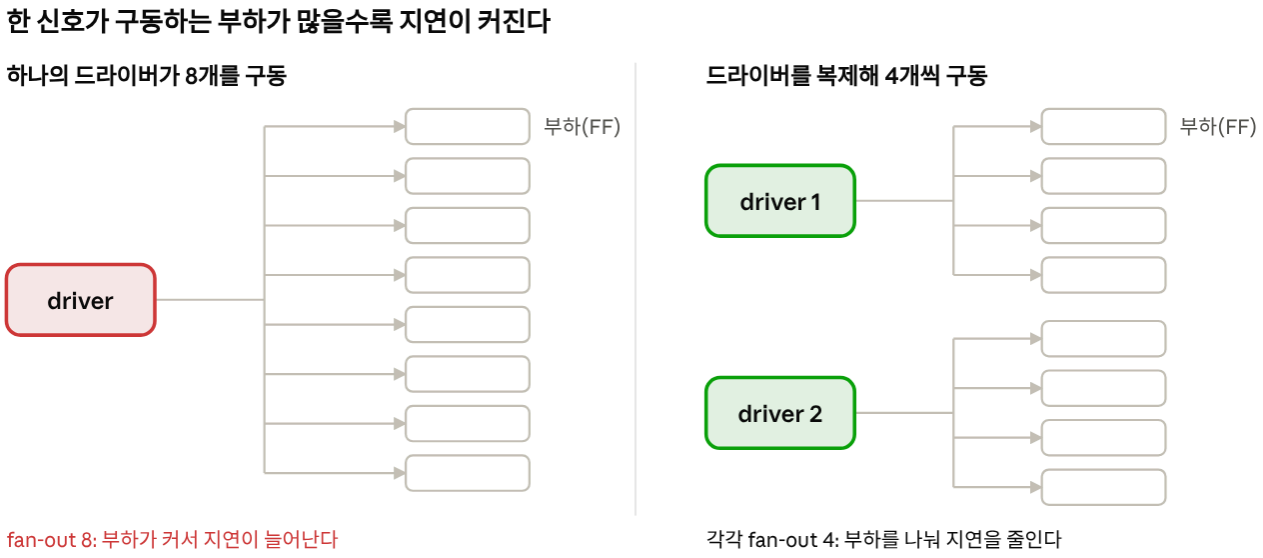
해결 방법.

- **동기식 설계**: 조합회로 출력을 FF에 넣어 클럭 엣지에서만 샘플링한다. 글리치는 엣지 사이에서 사라지므로 영향을 주지 못한다.
- 클럭이나 reset에는 조합회로 출력을 직접 쓰지 않고 FF 출력을 쓴다.
- 여러 비트가 동시에 바뀌지 않도록 Gray code를 쓴다.
- 논리식에 필요한 항(consensus term)을 추가해 hazard를 없앤다.

이 프로젝트에서. `tick_generator`의 `tick`은 `reg`로 선언해 FF 출력으로 내보내고, 버튼은 debounce를 거친 뒤 사용한다. `w_press`처럼 조합회로로 만든 펄스는 FF 입력에만 연결되어 글리치가 있어도 클럭 엣지에서 안정된 값만 반영된다.

4.4 Fan-out

정의. 하나의 게이트나 FF 출력이 동시에 구동하는 입력의 개수이다. 예를 들어 reset 신호 하나가 모든 FF의 reset 핀에 연결되면 fan-out은 FF 개수와 같다.



fan-out 8과 드라이버 복제 후 fan-out 4

왼쪽은 드라이버 하나가 부하 8개를 구동하는 경우, 오른쪽은 드라이버를 2개로 복제해 각각 4개씩 구동하는 경우이다.

원인과 문제점.

- 하나의 드라이버가 여러 부하를 구동하면 배선 길이와 부하 용량이 커져 신호가 느리게 바뀐다(전파 지연 증가).
- 지연이 커지면 setup time을 만족하지 못해 타이밍 violation이 생기고 최대 동작 주파수가 낮아진다.
- 부하가 많을수록 전력 소모도 늘어난다.
- FPGA에서는 reset, clock enable처럼 많은 FF가 공유하는 신호가 대표적인 high fan-out net이다.

해결 방법.

- **드라이버 복제(register duplication)**: 같은 로직을 여러 개 만들어 부하를 나눠 준다.
- **버퍼 트리**: 드라이버 뒤에 버퍼를 단계적으로 두어 부하를 분산한다.
- **전역 클럭 버퍼(BUFG) 사용**: 클럭처럼 부하가 큰 신호는 전용 배선을 쓴다.
- **툴 설정**: Vivado에서 `MAX_FANOUT` 속성이나 physical optimization(`phys_opt_design`)으로 자동 복제한다.

- **pipeline**: 긴 경로에 FF를 넣어 한 클럭에 건너야 하는 지연을 나눈다.

이 프로젝트에서. `reset` (SW15)가 debouncer, stopwatch, tick_generator, FND 모듈의 모든 FF에 연결된다. 회로가 작아 타이밍 문제는 없었지만, 큰 설계에서는 이런 공용 신호가 fan-out 문제의 원인이 될 수 있다.